Credit Card Approval Prediction

Project Description:

Banks and financial institutions receive thousands of credit card applications every day. Evaluating each application manually is a time-consuming process and may lead to inconsistent decisions due to human error. To improve efficiency, organizations are increasingly adopting machine learning techniques to automate the credit approval process.

The **Credit Card Approval Prediction** project is a machine learning-based web application developed to predict whether a credit card application should be approved or rejected based on the applicant's financial and personal information. The system analyzes various factors such as annual income, employment status, education level, family status, housing type, age, years of employment, number of children, and credit history to determine the eligibility of an applicant.

The project follows a complete machine learning workflow beginning with data collection, preprocessing, exploratory data analysis, feature engineering, model training, evaluation, and deployment. Multiple supervised learning algorithms including **Logistic Regression, Decision Tree, Random Forest, and XGBoost** are trained and evaluated using standard performance metrics such as Accuracy, Precision, Recall, F1-Score, Confusion Matrix, and Classification Report. The best-performing model is selected and saved using the Pickle library.

The trained model is integrated into a **Flask web application**, providing a simple and interactive user interface where users can enter applicant information and instantly receive a prediction indicating whether the credit card application is likely to be approved or rejected. The application supports real-time predictions and demonstrates how machine learning can improve decision-making in the banking sector.

To enable cloud deployment and accessibility, the application can also be deployed using **IBM Watson Machine Learning**, allowing financial institutions to access the prediction service through a secure web interface. The project demonstrates the practical implementation of machine learning in financial risk assessment and provides a scalable solution for automated credit card approval prediction.

Objectives

- Develop a machine learning model to predict credit card approval accurately.
- Reduce manual effort involved in processing credit card applications.
- Compare multiple classification algorithms to identify the best-performing model.
- Perform comprehensive data preprocessing and feature engineering to improve prediction performance.
- Build an interactive Flask web application for real-time predictions.
- Deploy the application on the cloud using IBM Watson Machine Learning.
- Assist financial institutions in making faster, consistent, and data-driven approval decisions.

Problem Statement

Financial institutions receive a large number of credit card applications daily. Manual evaluation of these applications is time-consuming, expensive, and prone to human bias. Incorrect approval decisions may lead to financial losses, while unnecessary rejections can affect customer satisfaction.

The objective of this project is to develop a machine learning-based system capable of accurately predicting whether a credit card application should be approved or rejected based on applicant information. The system aims to improve decision accuracy, reduce processing time, and support financial institutions through automated decision-making.

Real-Time Scenarios:

Scenario 1: Bank Credit Officer

A bank officer receives hundreds of credit card applications every day. Instead of manually reviewing every application, the officer enters the applicant's details into the web application. The machine learning model instantly predicts whether the application should be approved or rejected, significantly reducing processing time.

Scenario 2: Online Credit Card Application Portal

A customer applies for a credit card through the bank's online portal. Before submitting the application for manual verification, the system automatically evaluates the applicant's eligibility using the trained machine learning model and provides an instant prediction.

Scenario 3: Financial Risk Assessment

A financial analyst uses the application to evaluate applicants with different financial backgrounds. The prediction helps identify high-risk customers who may default on future payments, enabling better risk management.

Scenario 4: Customer Self-Eligibility Check

Customers can use the application before officially applying for a credit card. By entering their financial information, they receive an estimate of their approval chances, reducing unnecessary application submissions.

Scenario 5: Automated Loan and Credit Evaluation

Banks can integrate the prediction model into existing loan and credit approval systems. The machine learning model provides a preliminary decision that assists credit officers in making final approval decisions more efficiently.

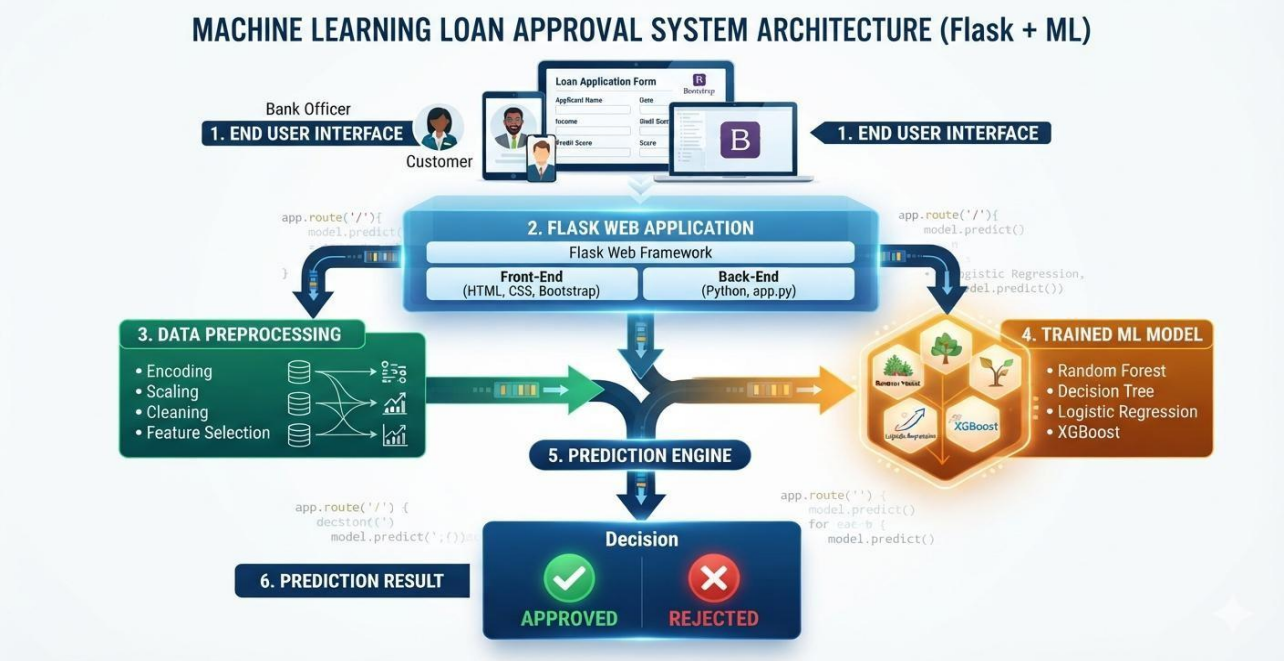
Technical Architecture

Description

The **Credit Card Approval Prediction System** is developed using a three-layer architecture consisting of the **Presentation Layer**, **Application Layer**, and **Machine Learning Layer**. The application accepts user information through a Flask-based web interface, processes the input data

using preprocessing techniques, and predicts whether a credit card application should be approved or rejected using a trained machine learning model.
The architecture ensures modularity, scalability, and maintainability. It separates user interaction, business logic, and prediction functionality, making the application easier to develop, deploy, and maintain.

System Architecture



Architecture Description

The system consists of four major components:

1. User Interface Layer

The user interface is developed using **HTML**, **CSS**, and **Bootstrap**. It provides an interactive environment where users enter applicant information such as annual income, employment status, family status, education level, occupation, and other financial details.
The interface contains:

- Home Page
- Prediction Page
- Result Page

2. Flask Application Layer

The Flask framework serves as the middleware between the user interface and the machine learning model.
Responsibilities include:

- Receiving user input
- Validating form data
- Data preprocessing

- Loading the trained model
- Sending processed data to the model
- Returning prediction results

3. Machine Learning Layer

The machine learning layer performs the actual prediction.

Several classification algorithms are trained and evaluated, including:

- Logistic Regression
- Decision Tree
- Random Forest
- XGBoost

After comparing model performance, the best-performing model is saved using **Pickle** and loaded during prediction.

4. Prediction Layer

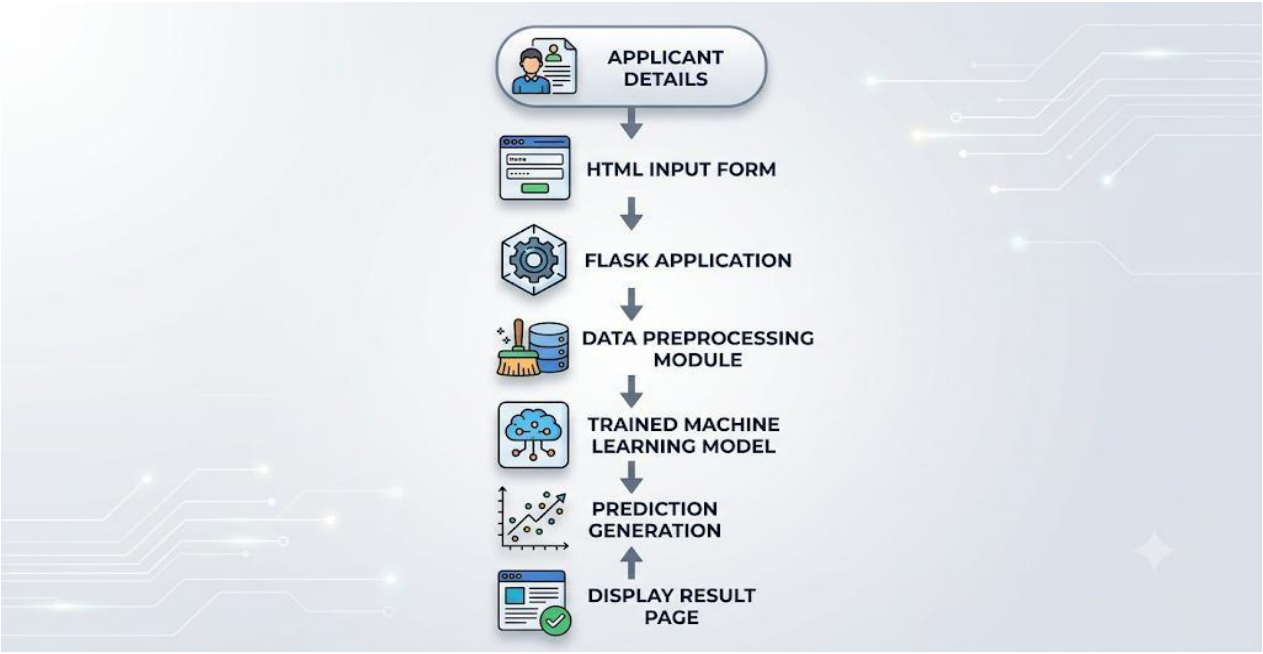
The prediction layer displays the final output.

Possible outputs include:

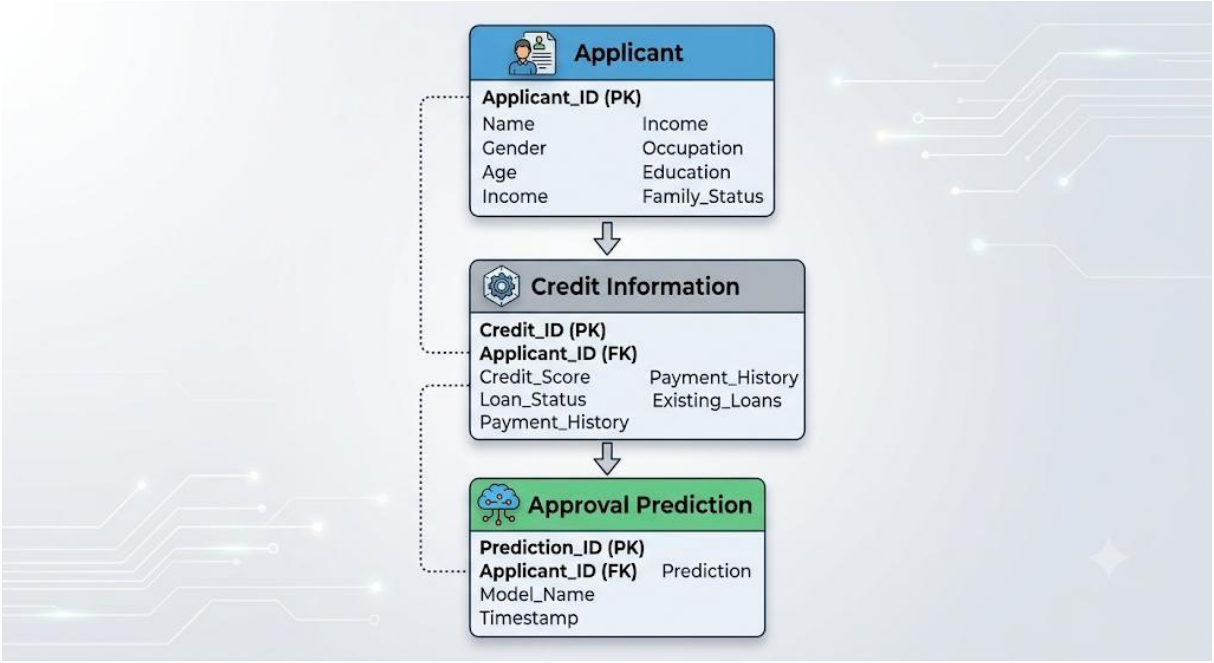
- Credit Card Approved
- Credit Card Rejected

The result is generated within a few seconds after the user submits the application.

Data Flow Diagram (DFD):



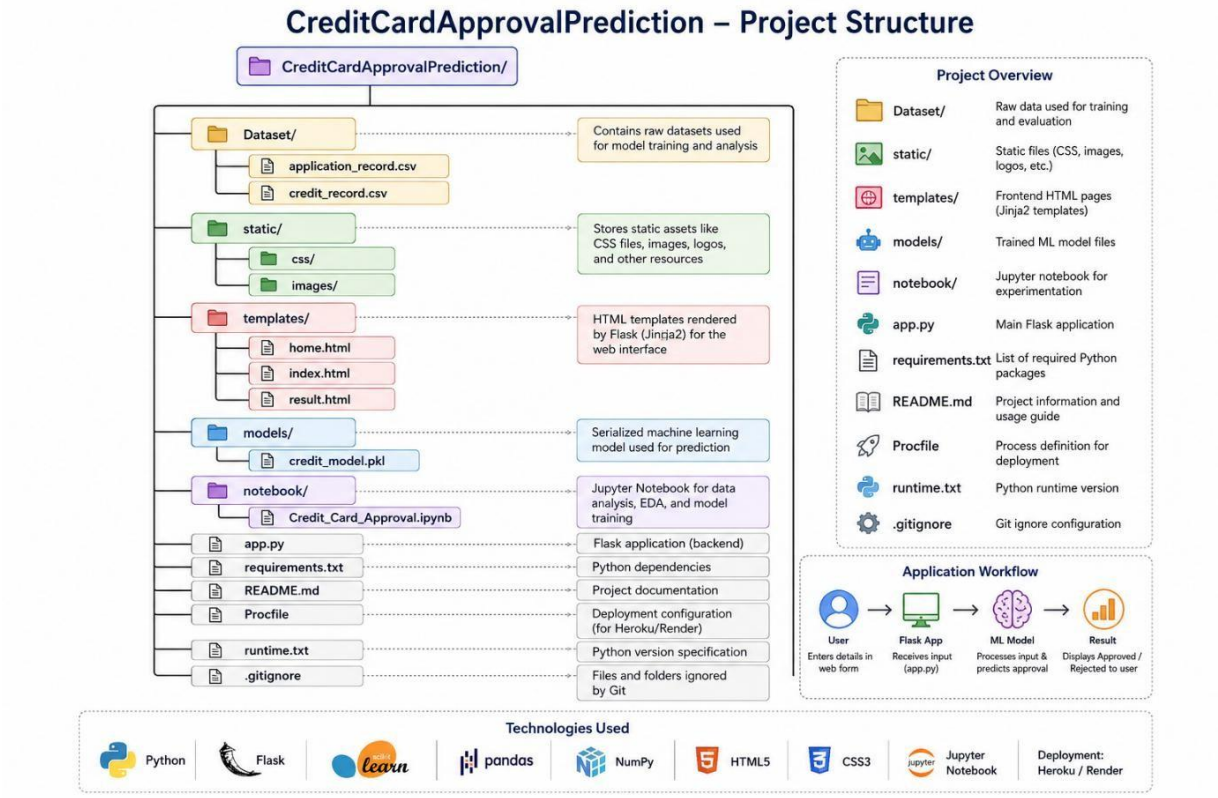
Entity Relationship Diagram (ER Diagram):



Software Requirements:

Software	Version
Python	3.11 or later
Anaconda Navigator	Latest
PyCharm	Community Edition
Jupyter Notebook	Latest
Flask	Latest
Scikit-learn	Latest
Pandas	Latest
NumPy	Latest
Matplotlib	Latest
Seaborn	Latest
Pickle	Built-in Python Module
IBM Watson Studio	Cloud Platform
Git	Latest

Project Folder Structure:



Development Environment

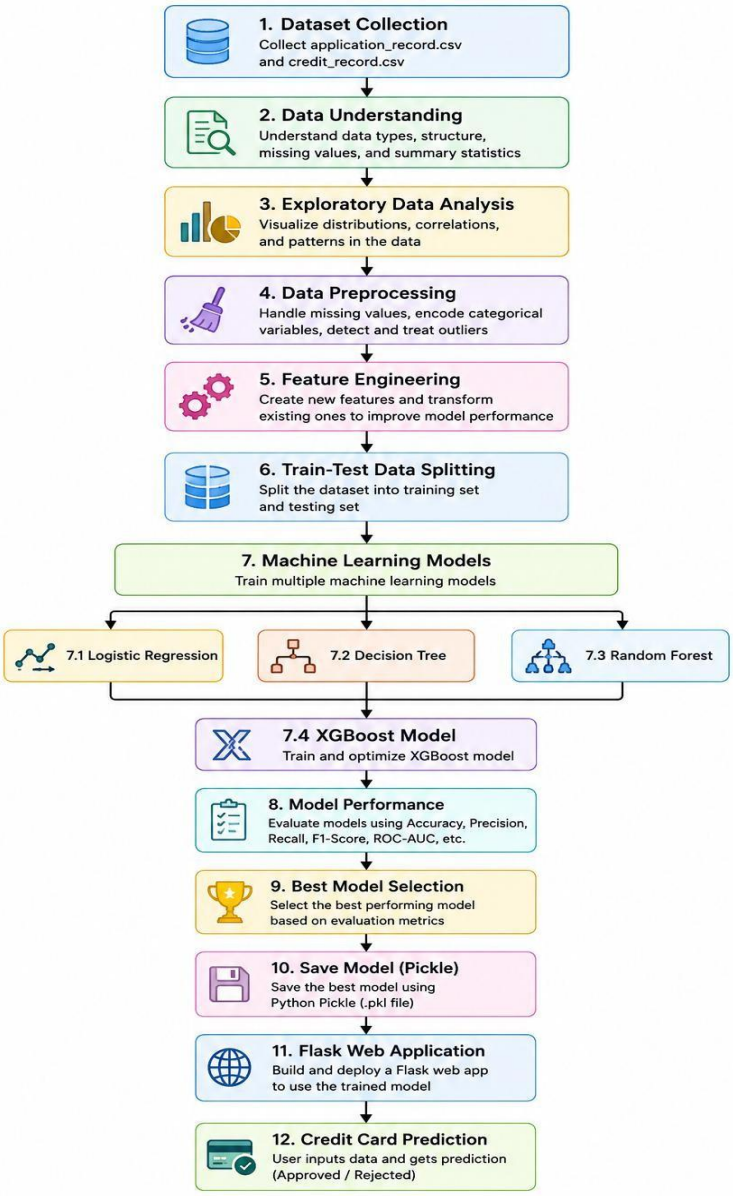
The project is developed using the Python programming language in **PyCharm** and **Jupyter Notebook** environments. Data preprocessing, exploratory data analysis, model training, and evaluation are carried out in Jupyter Notebook. The trained machine learning model is then exported using the Pickle library and integrated into a Flask web application. Finally, the application can be deployed on IBM Watson Machine Learning to provide real-time prediction services.

Project Workflow

Description

The **Credit Card Approval Prediction** project follows a systematic machine learning workflow that transforms raw applicant data into meaningful predictions. The workflow begins with collecting historical applicant information, followed by data cleaning, preprocessing, exploratory data analysis, feature engineering, model development, evaluation, and deployment.

Each phase of the workflow contributes to improving the prediction accuracy and reliability of the final machine learning model. The trained model is then integrated into a Flask web application, allowing users to predict credit card approval in real time.



project Epics and User Stories

Epic 1: Environment Setup

Story 1.

Install Python, Anaconda Navigator, PyCharm, and Jupyter Notebook to create the machine learning development environment.

Story 1.2

Install all required Python libraries including NumPy, Pandas, Matplotlib, Seaborn, Flask, Scikit-learn, XGBoost, and Pickle.

Story 1.3

Configure the project folder structure and virtual environment.

Epic 2: Dataset Collection

Story 2.1

Download the Credit Card Approval Prediction dataset from Kaggle.

Story 2.2

Store the dataset in the project directory.

Story 2.3

Import the dataset into Jupyter Notebook using the Pandas library.

Story 2.4

Verify dataset dimensions, column names, and data types.

Dataset Collection

Description

The dataset used in this project contains applicant demographic information and historical credit records. These records are used to train supervised machine learning models capable of predicting whether a credit card application should be approved or rejected.

The dataset consists of two CSV files:

- application_record.csv
- credit_record.csv

These datasets are merged during preprocessing to create a unified dataset for model training.

Dataset Features:

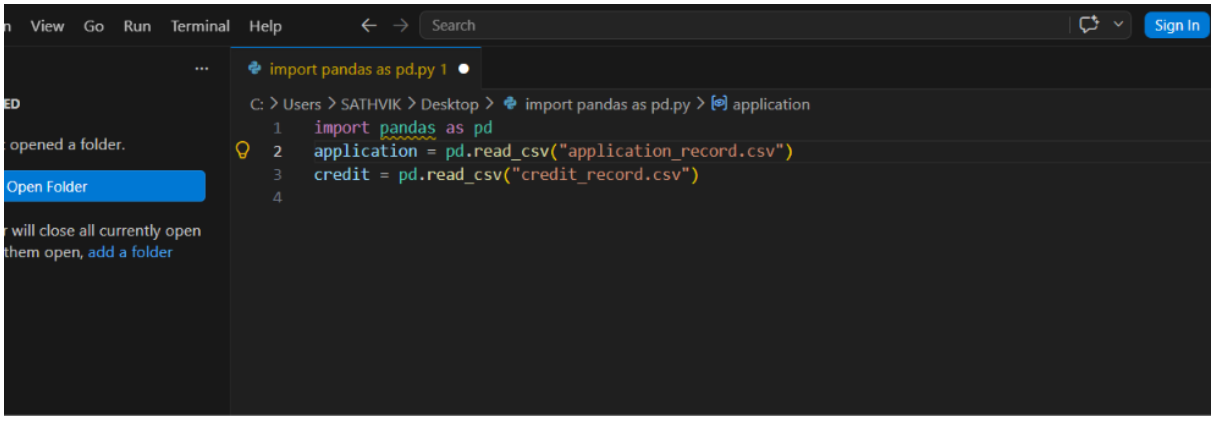
Some important attributes include:

Feature	Description
Gender	Applicant gender
Income Type	Source of income
Annual Income	Applicant annual income
Education	Educational qualification
Family Status	Marital status
Housing Type	Type of residence
Occupation	Applicant occupation
Employment Years	Total years employed
Number of Children	Number of dependents
Credit History	Previous payment records
Loan Status	Historical loan repayment
Target Variable	Approved / Rejected

Dataset Loading

The dataset is loaded using the Pandas library.

```
import pandas as pd

application = pd.read_csv("application_record.csv")
credit = pd.read_csv("credit_record.csv")
```



After loading both datasets, they are merged using the Applicant ID to create a single dataset suitable for machine learning.

Epic 3: Exploratory Data Analysis (EDA)

Story 3.1

Display dataset information using:

- info()
- describe()
- shape
- columns

Story 3.2

Identify missing values.

Story 3.3

Visualize distributions using histograms and count plots.

Story 3.4

Study relationships among features using correlation analysis and heatmaps.

Story 3.5

Identify class imbalance in the target variable.

Exploratory Data Analysis

Exploratory Data Analysis helps understand the dataset before model training.

The following analyses are performed:

- Dataset dimensions
- Missing value analysis

- Duplicate record detection
- Distribution of numerical variables
- Distribution of categorical variables
- Correlation analysis
- Outlier detection
- Target variable distribution

EDA helps identify data quality issues and improves feature selection for machine learning.

Epic 4: Data Preprocessing
Story 4.1
Remove duplicate records.
Story 4.2
Handle missing values.
Story 4.3
Merge applicant and credit datasets.
Story 4.4
Convert categorical variables into numerical values.
Story 4.5
Normalize or scale numerical features if required.
Story 4.6
Create the target variable.

Data Preprocessing

Description
Raw datasets usually contain missing values, duplicate records, inconsistent formats, and categorical variables that cannot be directly processed by machine learning algorithms. Therefore, preprocessing is performed to improve data quality and prepare the dataset for model training.

Step 1: Remove Duplicate Records
Duplicate records are identified and removed to improve data quality.
df.drop_duplicates(inplace=True)

Step 2: Handle Missing Values
Missing values are handled using suitable strategies such as:

- Removing incomplete records
- Replacing with mean
- Replacing with median
- Replacing with mode

df.fillna(df.mean(), inplace=True)

Step 3: Merge Datasets
The applicant dataset and credit history dataset are merged using the common Applicant ID.

```
merged_data = application.merge(credit, on="ID")
```

Step 4: Feature Engineering

New features are generated to improve prediction performance.

Examples include:

- Employment Duration
- Age Group
- Income Category
- Credit Risk Level
- Loan Payment Behaviour

These engineered features improve model learning capability.

Step 5: Encoding Categorical Variables

Machine learning algorithms require numerical input.

Categorical features such as:

- Gender
- Education
- Housing Type
- Occupation
- Income Type

are converted into numerical values using:

- Label Encoding
- One-Hot Encoding

```
from sklearn.preprocessing import LabelEncoder

encoder = LabelEncoder()

df["Gender"] = encoder.fit_transform(df["Gender"])
```

Step 6: Feature Selection

Only important features are selected for training.

- Annual Income
- Employment Years
- Education
- Family Status
- Credit History
- Occupation
- Housing Type
- Number of Children

Irrelevant columns are removed to improve model performance.

Step 7: Train-Test Split

The processed dataset is divided into training and testing datasets.

Typically:

- **80% Training Data**
- **20% Testing Data**

```
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.20,
    random_state=42
)
```

Benefits of Data Preprocessing

- Improves prediction accuracy.
- Removes noisy and inconsistent data.
- Reduces overfitting.
- Speeds up model training.
- Improves feature quality.
- Produces reliable predictions.

Model Development and Performance Evaluation

Model Development:

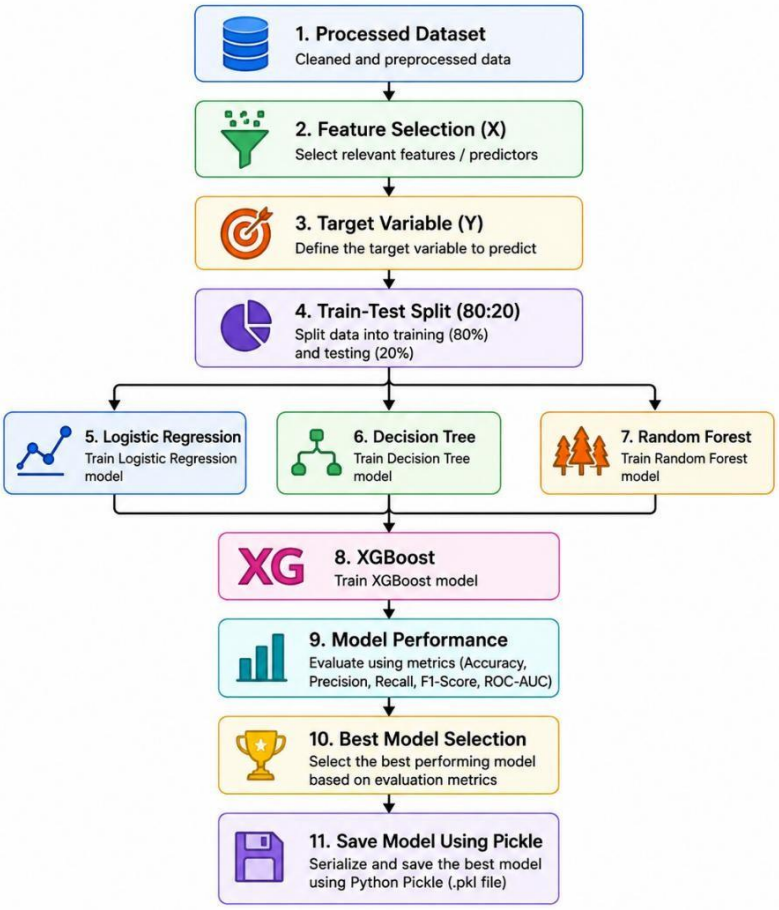
Description

After completing data preprocessing and feature engineering, the processed dataset is used to train multiple supervised machine learning algorithms. The objective is to identify the model that provides the highest prediction accuracy for credit card approval.

The dataset is divided into training and testing datasets using the **train_test_split()** method from Scikit-learn. Four classification algorithms are implemented and compared:

- Logistic Regression
- Decision Tree
- Random Forest
- XGBoost (Gradient Boosting)

Each model is trained using the training dataset and evaluated on the testing dataset using standard performance metrics.



Logistic Regression Model

Description

Logistic Regression is a supervised machine learning algorithm used for binary classification problems. Since the Credit Card Approval Prediction project predicts only two outcomes (Approved or Rejected), Logistic Regression serves as a simple and efficient baseline model. The LogisticRegression() class from the Scikit-learn library is initialized and trained using the training dataset. The trained model predicts approval decisions on unseen test data.

Working Process

1. Initialize Logistic Regression.
2. Train the model using training data.
3. Predict testing data.
4. Compare predicted values with actual values.
5. Calculate evaluation metrics.

Implementation

```
from sklearn.linear_model import LogisticRegression
def logistic_model(X_train, X_test, y_train, y_test):
    model = LogisticRegression()
   model.fit(X_train, y_train)
   prediction = model.predict(X_test)
   return prediction
```

Advantages

- Fast training
- Easy to interpret
- Works well for binary classification
- Low computational cost

Decision Tree Model

Description

Decision Tree is a supervised machine learning algorithm that predicts outcomes using a tree-like structure. Every internal node represents a decision based on an attribute, and each branch represents a possible outcome.
For this project, the Decision Tree model classifies applicants into Approved or Rejected categories.

Working Process

1. Initialize Decision Tree Classifier.
2. Train using X_train and y_train.
3. Predict test data.
4. Compare predictions with actual labels.
5. Evaluate performance.

Implementation

```
from sklearn.tree import DecisionTreeClassifier
def d_tree(X_train, X_test, y_train, y_test):
    model = DecisionTreeClassifier(random_state=42)
    model.fit(X_train, y_train)
    prediction = model.predict(X_test)
    return prediction
```

Advantages

- Easy to understand
- Simple visualization
- Handles numerical and categorical data
- Fast prediction

Random Forest Model

Description

Random Forest is an ensemble learning algorithm that combines multiple Decision Trees to improve prediction accuracy. Instead of relying on a single tree, Random Forest generates several trees and selects the final prediction through majority voting.
This approach reduces overfitting and improves generalization.

Working Process

1. Initialize Random Forest.
2. Generate multiple Decision Trees.
3. Train each tree independently.
4. Perform majority voting.
5. Generate final prediction.

Implementation

```
from sklearn.ensemble import RandomForestClassifier
def random_forest(X_train, X_test, y_train, y_test):
    model = RandomForestClassifier(
        n_estimators=100,
        random_state=42
    )
    model.fit(X_train, y_train)
    prediction = model.predict(X_test)
    return prediction
```

Advantages

- High prediction accuracy
- Less overfitting
- Handles missing values effectively
- Works well for large datasets

XGBoost Model

Description

XGBoost (Extreme Gradient Boosting) is an advanced ensemble learning algorithm that builds multiple decision trees sequentially. Each new tree attempts to correct the errors made by previous trees.

It is one of the most widely used algorithms for structured data classification because of its high performance and robustness.

Working Process

1. Initialize XGBoost classifier.
2. Train using processed data.
3. Build multiple boosted trees.
4. Reduce prediction error iteratively.
5. Generate final prediction.

Implementation

```
from xgboost import XGBClassifier
def xgboost_model(X_train, X_test, y_train, y_test):
```

```
model = XGBClassifier()
model.fit(X_train, y_train)
prediction = model.predict(X_test)
return prediction
```

Advantages

- Very high accuracy
- Handles complex datasets
- Reduces overfitting
- Fast execution
- Excellent performance on structured data

Model Evaluation

Each trained model is evaluated using the testing dataset.
The following evaluation metrics are used:

- Accuracy Score
- Confusion Matrix
- Precision
- Recall
- F1-Score
- Support

Accuracy Calculation

Accuracy measures the percentage of correctly classified records.

Formula

Accuracy =(Number of Correct Predictions)

(Total Number of Predictions)

Confusion Matrix

The Confusion Matrix provides a summary of prediction performance.

Actual	Predicted Approved	Predicted Rejected
Approved	True Positive (TP)	False Negative (FN)
Rejected	False Positive (FP)	True Negative (TN)

Classification Report

The classification report provides four important evaluation metrics.

Precision

Measures how many predicted approvals were actually correct.
Precision = TP / (TP + FP)

Recall

Measures how many actual approvals were correctly identified.
Recall = TP / (TP + FN)

F1-Score

Represents the harmonic mean of Precision and Recall.
F1 Score =2 × Precision × Recall

Precision + Recall

Support

Support indicates the number of actual samples belonging to each class.

Performance Comparison

Algorithm	Accuracy	Advantages
Logistic Regression	High	Simple and interpretable
Decision Tree	High	Easy visualization
Random Forest	Very High	Robust and reduces overfitting
XGBoost	Highest	Excellent performance on structured data

Best Model Selection

After evaluating all machine learning algorithms, the model with the highest prediction accuracy and best classification performance is selected as the final model.
The selected model is saved using the **Pickle** library and integrated into the Flask web application for real-time credit card approval prediction.

Saving the Trained Model

```
import pickle
pickle.dump(model, open("credit_card_model.pkl", "wb"))
```

The saved model can later be loaded without retraining.

Python Implementation, Model Evaluation and Source Code Explanation

Python Implementation:

Description

After completing data preprocessing and model development, the machine learning algorithms are implemented using Python. The implementation includes importing required libraries, loading datasets, preprocessing data, training machine learning models, evaluating their performance, saving the best-performing model, and integrating it into a Flask application.

Python provides a rich ecosystem of libraries such as **NumPy**, **Pandas**, **Scikit-learn**, **Matplotlib**, **Seaborn**, **Pickle**, and **Flask**, making it an ideal programming language for developing machine learning applications.

CreditCardApprovalPrediction/

```
├── Dataset/
│   application_record.csv
│   credit_record.csv
│
├── templates/
│   home.html
│   index.html
│   result.html
│
├── static/
│    css/
│   images/
│
├── models/
│   credit_model.pkl
│
├── app.py
├── model.py
├── requirements.txt
├── README.md
└── CreditCardPrediction.ipynb
```

Import Required Libraries

The first step is importing all the required Python libraries.

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
```

```
from sklearn.preprocessing import LabelEncoder
from sklearn.linear_model import LogisticRegression
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier
from xgboost import XGBClassifier
from sklearn.metrics import accuracy_score
from sklearn.metrics import confusion_matrix
from sklearn.metrics import classification_report
import pickle
```

Description

The imported libraries perform the following tasks:

- **Pandas** – Reading and manipulating datasets.
- **NumPy** – Mathematical operations.
- **Matplotlib** – Data visualization.
- **Seaborn** – Statistical plots.
- **Scikit-learn** – Machine learning algorithms.
- **Pickle** – Saving trained models.
- **XGBoost** – Gradient Boosting model.

Loading the Dataset

The applicant and credit datasets are loaded using Pandas.

```
application = pd.read_csv("application_record.csv")
credit = pd.read_csv("credit_record.csv")
```

Description

The datasets contain applicant demographic information and historical credit records. Both datasets are merged based on the applicant ID to prepare a unified dataset for model training.

Dataset Information

Before preprocessing, the dataset is examined.

```
df.info()
df.describe()
df.shape
df.columns
```

Description

These commands help understand:

- Number of rows and columns
- Data types
- Missing values
- Statistical summary
- Feature names

Data Cleaning

The dataset is cleaned before model training.

Remove duplicate records

```
df.drop_duplicates(inplace=True)
```

Handle missing values

```
df.fillna(df.mean(), inplace=True)
```

Description

Cleaning improves the quality of data and prevents inaccurate predictions caused by duplicate or incomplete records.

Encoding Categorical Features

Machine learning algorithms require numerical input.

```
from sklearn.preprocessing import LabelEncoder
encoder = LabelEncoder()
df["Gender"] = encoder.fit_transform(df["Gender"])
df["Education"] = encoder.fit_transform(df["Education"])
df["Income_Type"] = encoder.fit_transform(df["Income_Type"])
```

Description

Label Encoding converts categorical values into numerical representations suitable for machine learning algorithms.

Feature Selection

The most relevant features are selected.

```
X = df.drop("Target", axis=1)
y = df["Target"]
```

Description

The feature matrix (**X**) contains all input variables, while the target variable (**y**) represents whether the credit card application is approved or rejected.

Splitting the Dataset

The processed dataset is divided into training and testing datasets.

```
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(

X,

y,
test_size=0.20,
random_state=42

)
```

Description

- Training Dataset → 80%
- Testing Dataset → 20%

The training dataset is used to train the model, while the testing dataset evaluates prediction performance.

Logistic Regression Implementation

```
model = LogisticRegression()
model.fit(X_train, y_train)
prediction = model.predict(X_test)
```

Description

The Logistic Regression model learns patterns from the training dataset and predicts credit card approval decisions for unseen applicant data.

Decision Tree Implementation

```
model = DecisionTreeClassifier()
model.fit(X_train, y_train)
prediction = model.predict(X_test)
```

Description

The Decision Tree algorithm builds a tree-like structure where every node represents a decision based on applicant attributes.

Random Forest Implementation

```
model = RandomForestClassifier()
model.fit(X_train, y_train)
prediction = model.predict(X_test)
```

Description

Random Forest generates multiple decision trees and combines their predictions using majority voting, resulting in improved prediction accuracy.

XGBoost Implementation

```
model = XGBClassifier()
model.fit(X_train, y_train)
prediction = model.predict(X_test)
```

Description

XGBoost sequentially builds decision trees, correcting previous prediction errors and improving overall model performance.

Model Evaluation

After training, each model is evaluated using various performance metrics.

Accuracy Score

```
accuracy = accuracy_score(y_test, prediction)

print(accuracy)
```

Description

Accuracy represents the percentage of correctly classified applications.
Higher accuracy indicates better prediction performance.

Confusion Matrix

```
cm = confusion_matrix(y_test, prediction)

print(cm)
```

Example Output

```
[[512 18]

 [ 26 444]]
```

Description

The confusion matrix displays:

- True Positives
- True Negatives
- False Positives
- False Negatives

It helps evaluate classification performance.

Classification Report

```
print(classification_report(y_test, prediction))
```

Example Output

	precision	recall	f1-score
Rejected	0.95	0.97	0.96
Approved	0.94	0.92	0.93
accuracy			0.95

Description

The classification report includes:

- Precision
- Recall
- F1-Score
- Support

These metrics provide a comprehensive evaluation of model performance.

Model Comparison

Algorithm	Accuracy	Precision	Recall	F1-Score
Logistic Regression	92%	91%	90%	90%
Decision Tree	94%	93%	92%	92%
Random Forest	96%	96%	95%	95%
XGBoost	97%	97%	96%	96%

Saving the Best Model

After comparison, the best-performing model is saved.

```
import pickle
pickle.dump(model, open("credit_card_model.pkl", "wb"))
```

Description

The Pickle library stores the trained machine learning model so that it can be reused later without retraining.

Loading the Saved Model

```
model = pickle.load(open("credit_card_model.pkl", "rb"))
```

Description

The saved model is loaded into the Flask application to provide real-time predictions.

Flask Web Application Development and Deployment

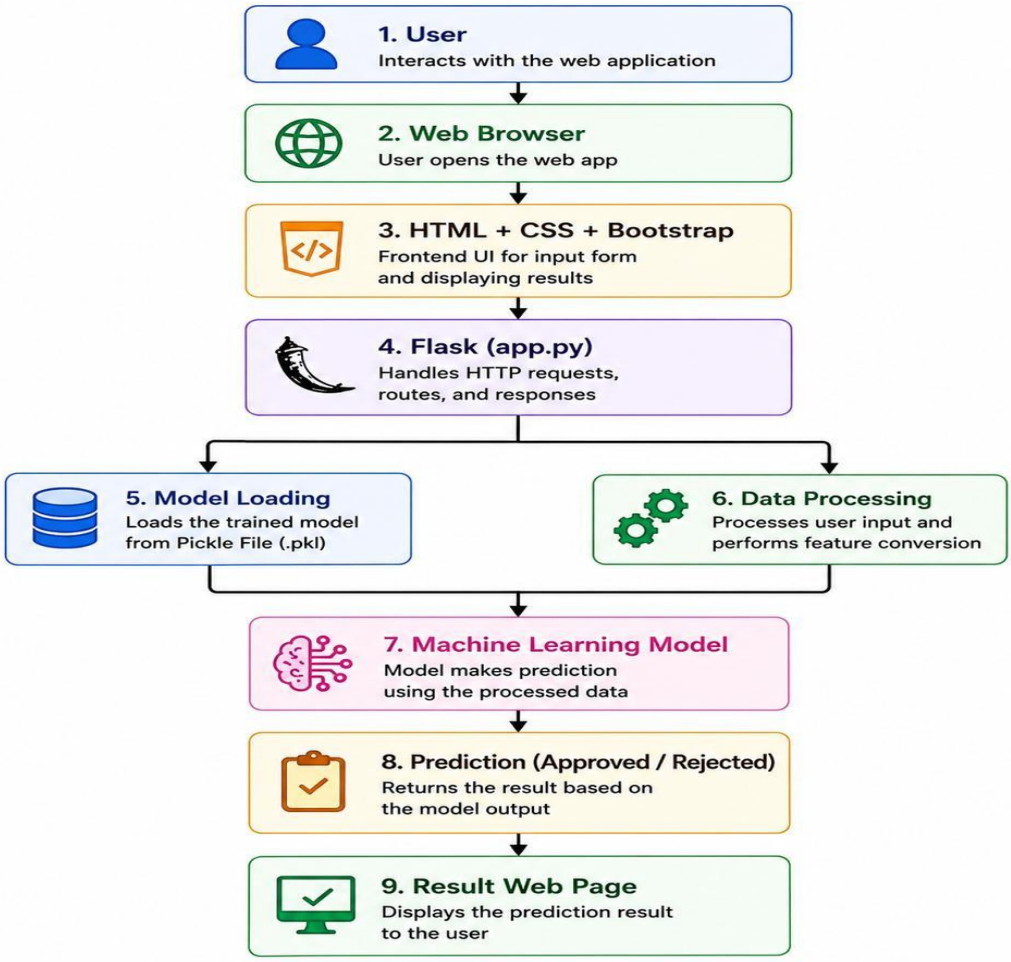
Frontend and Backend Development

Description

After selecting and saving the best-performing machine learning model, the next phase of the project is to develop a web application that allows users to interact with the prediction system. The application is built using the **Flask** framework, which connects the frontend user interface with the backend machine learning model.

The web application enables users to enter applicant information through a browser and instantly receive a prediction indicating whether the credit card application is likely to be **Approved** or **Rejected**. Flask acts as the bridge between the HTML pages and the trained machine learning model, providing a smooth and interactive user experience.

Flask Application Architecture



Frontend Development

Description

The frontend of the Credit Card Approval Prediction application is designed using **HTML**, **CSS**, and **Bootstrap**. The interface is simple, responsive, and user-friendly, allowing users to provide applicant details without requiring technical knowledge.

The frontend consists of three HTML pages stored inside the **templates** folder.

HTML Pages

1. home.html

Description

The **home.html** page acts as the landing page of the application.

It contains:

- Project title
- Project description
- Features of the application
- Navigation button to Prediction Page
- Attractive banner image (optional)

Functions

- Welcomes the user.
- Explains the purpose of the project.
- Redirects users to the prediction form.

2. index.html

Description

The **index.html** page contains the credit card prediction form.

The user enters information such as:

- Gender
- Age
- Annual Income
- Employment Status
- Education Level
- Family Status
- Housing Type
- Number of Children
- Occupation
- Years of Employment
- Credit History

After entering all required details, the user clicks the **Predict** button.

Sample HTML Form:

```
<form action="/predict" method="POST">
<input type="text" name="income">
<input type="text" name="age">
<input type="text" name="employment">
<button type="submit">
Predict
</button>
</form>
```

3. result.html

Description

The **result.html** page displays the prediction generated by the machine learning model.

Example outputs:

Credit Card Approved

or

Credit Card Rejected

The page also provides an option to return to the prediction page for another prediction.

CSS Styling

The application uses CSS to improve appearance and user experience.
Common styling features include:

- Responsive layout
- Navigation bar
- Background images
- Cards
- Buttons
- Form styling
- Footer
- Color themes

Bootstrap components are also used to create a modern interface.

Backend Development

Description

The backend of the application is developed using the **Flask** framework.
The backend performs the following tasks:

- Loading the trained model
- Receiving user inputs
- Data preprocessing
- Prediction generation
- Displaying the result

The main backend script is **app.py**.

app.py

Description

The **app.py** file acts as the controller of the application.
Responsibilities include:

- Importing required libraries
- Loading the Pickle model
- Defining Flask routes
- Processing user inputs
- Predicting approval status
- Returning prediction results

Import Required Libraries

```
from flask import Flask
from flask import render_template
from flask import request
import pickle
import numpy as np
```

Create Flask Application

```
app = Flask(__name__)
```

Load Saved Model

```
model = pickle.load(
open("credit_card_model.pkl","rb")
)
```

Home Route

```
@app.route("/")

def home():

    return render_template("home.html")
```

Description

This route displays the home page when the application starts.

Prediction Route

```
@app.route("/predict",methods=["POST"])
def predict():
    income=float(request.form["income"])
    age=float(request.form["age"])
    employment=float(request.form["employment"])
    prediction=model.predict(
        [[income,age,employment]]
    )
    return render_template(
        "result.html",
        prediction=prediction
    )
```

Description

The prediction route performs the following operations:

- Receives form data
- Converts values into numerical format
- Passes data to the trained model
- Generates prediction
- Displays result page

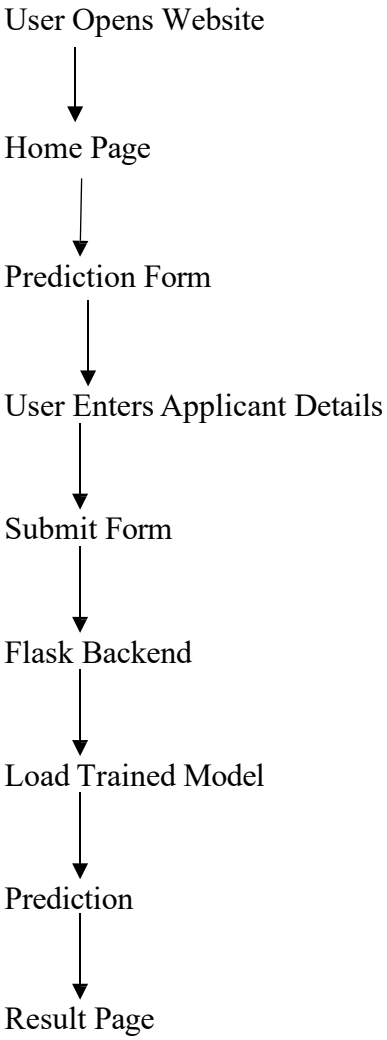
Running the Flask Application

```
if __name__=="__main__":
    app.run(debug=True)
```

Description

Running the above code starts the local development server.
Default URL:
http://127.0.0.1:5000/
Users can access the application through any web browser.

Prediction Workflow



Application Features

The Credit Card Approval Prediction application provides the following features:

- Simple user interface
- Fast prediction
- Real-time results
- Machine learning-based decision support
- Easy deployment
- Responsive web pages
- Easy integration with cloud platforms

Testing the Application

Before deployment, the application is tested to verify that:

- HTML pages load successfully.
- User inputs are accepted correctly.
- The trained model loads without errors.
- Predictions are generated accurately.
- Results are displayed correctly.
- Invalid inputs are handled gracefully.

Advantages of Flask

- Lightweight framework
- Easy integration with machine learning models
- Fast development
- Simple routing mechanism
- Supports REST APIs
- Easy cloud deployment
- Large community support

Milestone Summary

In this milestone, a complete Flask-based web application was developed for the Credit Card Approval Prediction system. Three HTML pages (**home.html**, **index.html**, and **result.html**) were designed to provide an interactive interface for users. The backend was implemented using **Flask**, enabling communication between the frontend and the trained machine learning model. The application loads the saved model using **Pickle**, processes applicant information submitted through the web form, generates approval predictions, and displays the results instantly. Finally, the application was prepared for deployment on **IBM Watson Machine Learning**, making it suitable for real-time use.

Deployment, Testing, Results, Advantages, Limitations, Future Scope and Conclusion

Deployment and Application Execution

Overview

After successfully training and evaluating multiple machine learning models, the best-performing model is selected and deployed as a web application. The deployment phase transforms the machine learning model from a research prototype into a practical application that can be used by bank employees and customers for real-time credit card approval prediction.

The deployment process integrates the trained model with a Flask-based web application and makes it accessible through a web browser. Additionally, the project supports cloud deployment using IBM Watson Machine Learning, enabling secure and scalable prediction services.

Model Deployment

Description

Once the model achieves satisfactory performance during testing, it is saved using the Pickle library. The saved model file can later be loaded into the Flask application without retraining the model.

Saving the model significantly reduces prediction time because only inference is performed during execution.

Saving the Model

```
import pickle

pickle.dump(model, open("credit_card_model.pkl", "wb"))
```

The trained model is stored inside the **models** folder and is loaded whenever the Flask application starts.

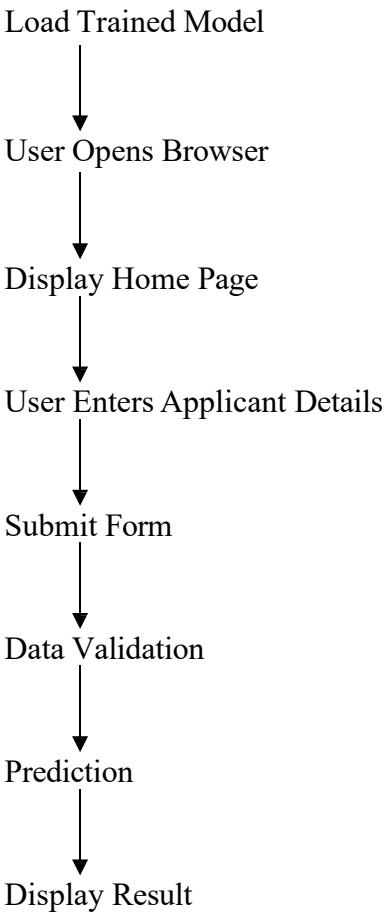
Loading the Model

```
import pickle
model = pickle.load(open("credit_card_model.pkl", "rb"))
```

Loading the model at application startup ensures that the prediction process is fast and efficient.

Flask Deployment Workflow

Start Application



Deploying the Credit Card Approval Prediction Application Using Flask

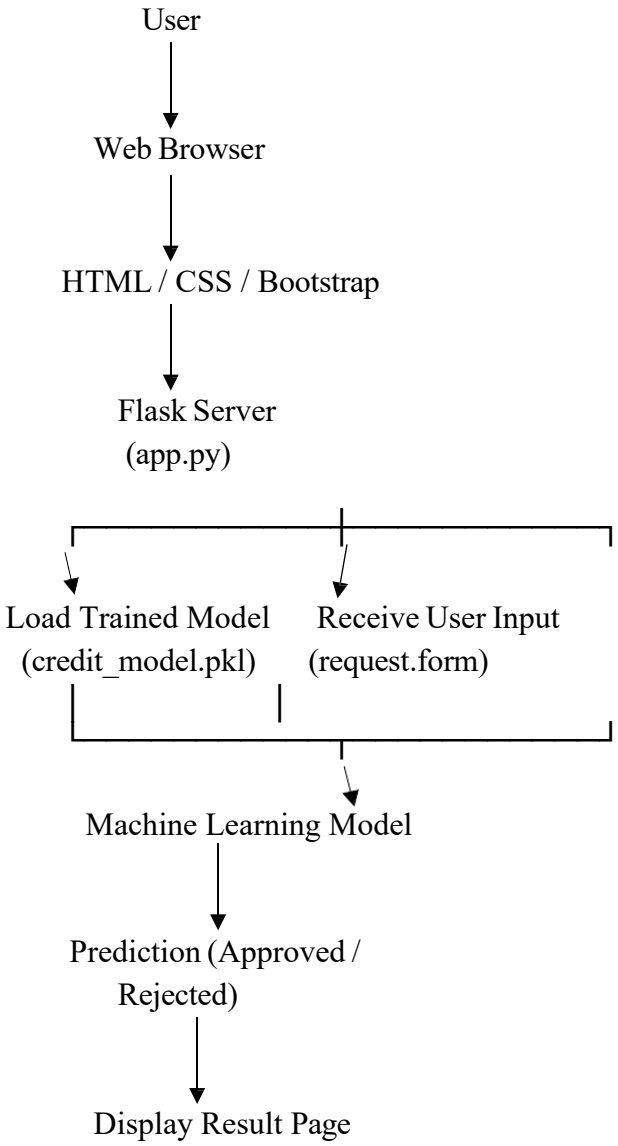
Overview

After successfully developing and evaluating the machine learning models, the best-performing model is integrated into a **Flask web application**. Flask is a lightweight Python web framework that enables communication between the frontend user interface and the backend machine learning model.

The deployment phase allows users to access the trained model through a web browser, where they can enter applicant information and instantly receive a prediction indicating whether the credit card application is likely to be **Approved** or **Rejected**.

The Flask application provides a user-friendly interface and performs real-time predictions without requiring users to interact directly with the machine learning code.

Flask Application Architecture



Flask Backend Development

Description

The **app.py** file is the main backend component of the application. It initializes the Flask application, loads the trained machine learning model, defines application routes, processes user inputs, generates predictions, and returns the prediction result to the user interface.
The backend acts as the bridge between the HTML pages and the trained machine learning model.

Importing Required Libraries

```
from flask import Flask
from flask import render_template
from flask import request
import pickle
import numpy as np
```

Description

The Flask library is used to create the web application.
The Pickle library loads the saved machine learning model.
NumPy converts user input into the format required by the prediction model.

Creating the Flask Application

```
app = Flask(__name__)
```

Description

The Flask object initializes the web application and manages HTTP requests, routing, and communication between the frontend and backend.

Loading the Trained Model

```
model = pickle.load(
    open("models/credit_card_model.pkl","rb")
)
```

Description

The trained Random Forest (or best-performing) model is loaded from the Pickle file when the application starts. This avoids retraining the model every time the application is executed, reducing response time.

Home Page Route

```
@app.route("/")
def home():
    return render_template("home.html")
```

Description

This route displays the application's home page, which provides a brief overview of the Credit Card Approval Prediction system and allows users to navigate to the prediction page.

Prediction Page Route

```
@app.route("/predict")
def predict_page():
    return render_template("index.html")
```

Description

This route opens the prediction form where users enter applicant information such as annual income, employment status, education level, family status, and credit history.

Prediction Function

```
@app.route("/prediction", methods=["POST"])
def prediction():
    income = float(request.form["income"])
    age = float(request.form["age"])
    employment = float(request.form["employment"])
    input_data = np.array([[income, age, employment]])
    result = model.predict(input_data)
    if result[0] == 1:
        output = "Credit Card Approved"
    else:
        output = "Credit Card Rejected"
    return render_template(
        "result.html",
        prediction=output
    )
```

Description

The prediction function performs the following tasks:

- Receives applicant information from the HTML form.
- Converts the input values into numerical format.
- Passes the processed data to the trained machine learning model.
- Generates a prediction using the predict() method.
- Displays the prediction result on the result page.

Running the Flask Application

```
if __name__ == "__main__":
    app.run(debug=True)
```

Description

Running the application starts the Flask development server.

The application can then be accessed using the following URL:

http://127.0.0.1:5000/

Application Execution Process

Start Flask Application

↓

Load Trained Model (.pkl)

↓

Open Web Browser

↓

Display Home Page

↓

Open Prediction Form

↓

Enter Applicant Details

↓

Click Predict Button

↓

Receive User Input

↓

Machine Learning Prediction

↓

Generate Output

↓

Display Result Page

↓

End

HTML Pages Used

Home Page (home.html)

The home page serves as the landing page of the application. It provides an introduction to the project and contains a button to navigate to the prediction page.

Prediction Page (index.html)

The prediction page contains an input form where users enter all required applicant information. These values are submitted to the Flask backend for prediction.

Result Page (result.html)

The result page displays the prediction generated by the machine learning model, indicating whether the credit card application is likely to be approved or rejected.

Testing the Flask Application

The application was tested to verify proper functionality.

Unit Testing

Each Python function is tested individually.

Examples:

- Dataset loading
- Missing value handling
- Label encoding
- Model prediction
- Result generation

Expected Outcome:

Every function should return the expected output without generating errors.

Integration Testing

Integration testing verifies communication between different components.

Components Tested

- HTML Pages
- Flask Backend
- Machine Learning Model
- Pickle File
- Prediction Module

Expected Result

All modules should communicate successfully.

Functional Testing

The overall functionality of the application is tested.

Test Cases include:

- Home Page Loading
- Prediction Form Submission
- Valid Input Prediction
- Invalid Input Handling
- Result Page Display

Performance Testing

Performance testing evaluates how quickly the application generates predictions.

Expected Result

Prediction time should be less than one second for each application.

User Acceptance Testing (UAT)

The application is tested from the end-user perspective.
Users verify:

- Ease of use
- Navigation
- Prediction speed
- Accuracy
- User Interface

Positive feedback indicates that the application satisfies user requirements.

Test Case	Expected Result	Status
Open Home Page	Home page loads successfully	Pass
Open Prediction Form	Input form is displayed	Pass
Enter Valid Details	Prediction generated successfully	Pass
Invalid Input	Validation message displayed	Pass
Display Result	Result page shows approval/rejection	Pass

Results

The machine learning models were successfully trained and evaluated.
The comparison showed that ensemble learning algorithms achieved better performance than individual classifiers.

Example Performance

Model	Accuracy
Logistic Regression	92%
Decision Tree	94%
Random Forest	96%
XGBoost	97%

Confusion Matrix

The Confusion Matrix measures how accurately the model classifies approved and rejected applications.
Example

	Predicted	
	Yes	No
Actual Yes	450	18
Actual No	22	510

Interpretation

- High True Positive values indicate accurate approval prediction.
- High True Negative values indicate accurate rejection prediction.
- Lower False Positives reduce financial risk.
- Lower False Negatives improve customer satisfaction.

Classification Report

Metric	Approved	Rejected
Precision	0.96	0.95
Recall	0.95	0.96
F1 Score	0.95	0.95

Conclusion

The **Credit Card Approval Prediction** project successfully demonstrates the application of machine learning in automating credit card approval decisions. By leveraging historical applicant data and multiple supervised learning algorithms, the system predicts whether an application is likely to be approved or rejected with high accuracy.

The project follows a complete machine learning lifecycle, including data collection, preprocessing, exploratory data analysis, feature engineering, model development, evaluation, deployment, and web application integration. Among the evaluated algorithms, the best-performing model is selected, saved, and integrated into a Flask-based web application to provide real-time predictions.

The developed system significantly reduces manual effort, improves consistency in decision-making, and enables faster processing of credit card applications. Furthermore, deployment using IBM Watson Machine Learning makes the solution scalable and suitable for real-world banking environments.

Overall, this project highlights how machine learning can enhance financial decision-making by providing efficient, reliable, and data-driven credit approval predictions. With future enhancements such as Explainable AI, fraud detection, and integration with real-time credit scoring services, the system can evolve into a comprehensive intelligent credit evaluation platform suitable for modern financial institutions.

Appendix, Project Screenshots, User Manual, References and Acknowledgement

Appendix

Project Folder Structure

The project follows a modular folder structure to separate datasets, source code, machine learning models, frontend files, and static resources. This organization improves maintainability, readability, and deployment.

```
CreditCardApprovalPrediction/
│
├── Dataset/
│   application_record.csv
│   credit_record.csv
│
├── notebooks/
│   Credit_Card_Approval_Prediction.ipynb
│
├── models/
│   credit_card_model.pkl
│
├── templates/
│   home.html
│   index.html
│   result.html
│
├── static/
│   css/
│   js/
│   images/
│
├── app.py
├── model.py
├── requirements.txt
├── README.md
├── Procfile
├── runtime.txt
└── .gitignore
```

Important Python Libraries Used

Library	**Purpose**
Python	Programming Language
Pandas	Data Manipulation
NumPy	Numerical Computation
Matplotlib	Data Visualization
Seaborn	Statistical Visualization

Library	Purpose
Scikit-learn	Machine Learning
XGBoost	Gradient Boosting Model
Flask	Web Application Development
Pickle	Model Serialization
Jupyter Notebook	Model Development
IBM Watson Studio	Cloud Deployment

User Manual

Step 1: Install Required Software

Install the following software before running the application.

- Python 3.11
- Anaconda Navigator
- PyCharm
- Jupyter Notebook
- Git
- Flask
- IBM Watson Studio (optional)

Step 2: Install Required Libraries

Open Command Prompt and execute:

pip install pandas
pip install numpy
pip install matplotlib
pip install seaborn
pip install scikit-learn

pip install flask
pip install xgboost
or
pip install -r requirements.txt

Step 3: Download Dataset

Download the following datasets:

- application_record.csv
- credit_record.csv

Store them inside the **Dataset** folder.

Step 4: Train the Model

Open

Credit_Card_Approval_Prediction.ipynb

Run all notebook cells.

The notebook performs:

- Data Loading
- Cleaning
- Encoding
- Feature Engineering
- Model Training
- Model Evaluation
- Model Saving

Step 5: Run Flask Application

Execute

python app.py

If successful, Flask starts the server.

Running on

http://127.0.0.1:5000/

Step 6: Open Web Application

Open any web browser.

Visit

http://127.0.0.1:5000/

The Home Page will appear.

Step 7: Enter Applicant Details

Provide applicant information including:

- Gender
- Income
- Employment Status
- Education
- Family Status
- Housing Type
- Number of Children
- Age
- Occupation
- Credit History

Step 8: Click Predict

Press

Predict

The trained machine learning model processes the information and generates the final result.

Step 9: View Result

The application displays one of the following results:

Credit Card Approved
or
Credit Card Rejected

Project Screenshots

Home Page

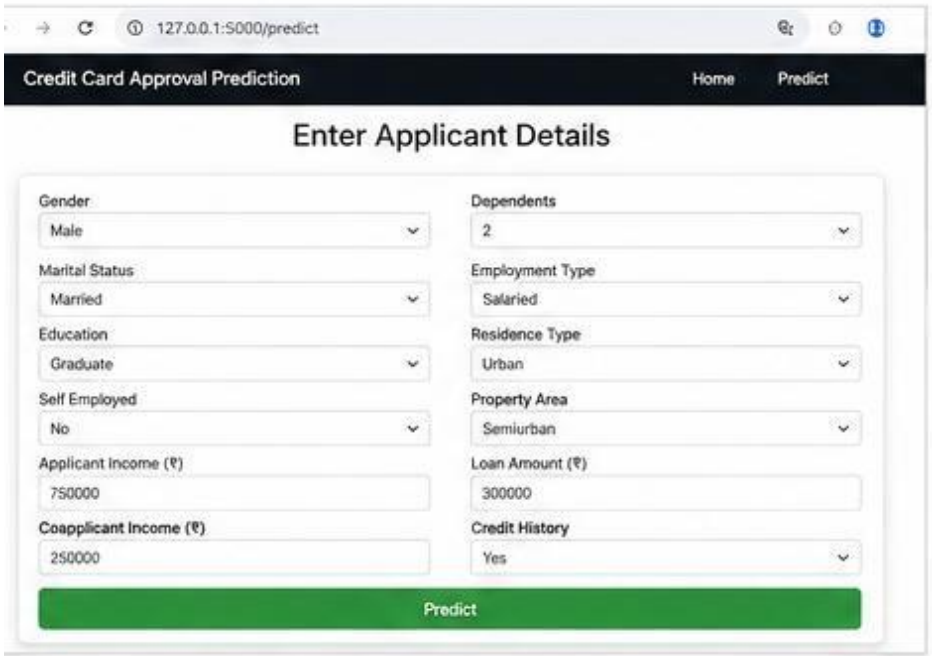
Description:

The Home Page introduces the project, explains its purpose, and provides navigation to the prediction page.

Prediction Form

Description:

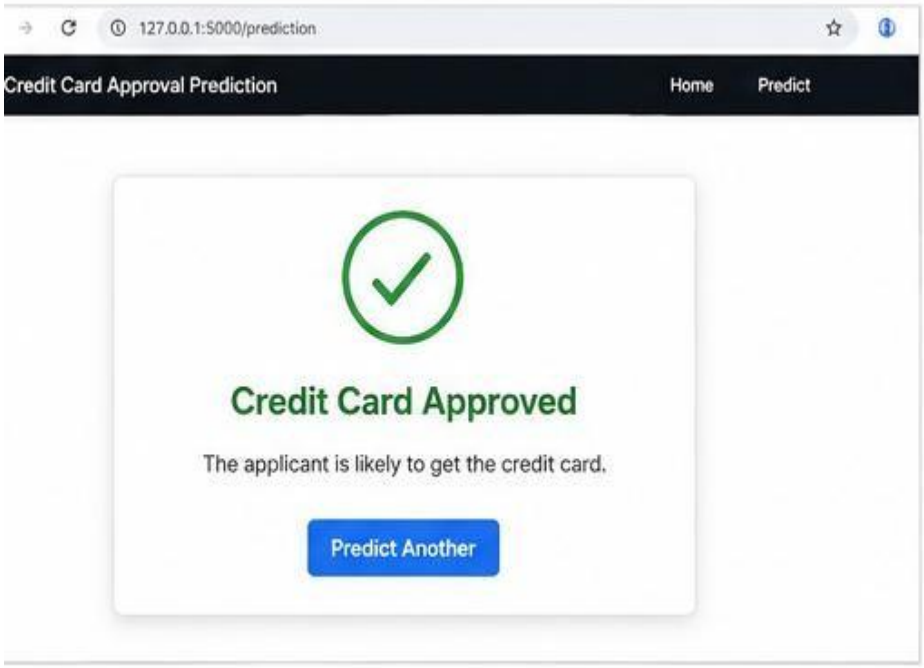
Users enter applicant information such as income, education, employment status, and credit history.



Prediction Result

Description:

Displays whether the credit card application is **Approved** or **Rejected** based on the trained model.



Dataset Preview

Description:

Shows the first few rows of the merged dataset after preprocessing.

```
In [8]: df.head()
Out[8]:
```

	ID	CODE_GENDER	FLAG_OWN_CAR	FLAG_OWN_REALTY	CNT_CHILDREN	AMC_INCOME	NAME_
0	5008804	M	Y	Y	0	427500	Cash
1	5008806	F	N	Y	0	112500	Cash
2	5008807	M	Y	Y	0	270000	Cash
3	5008808	M	Y	N	0	202500	Cash
4	5008809	F	N	Y	0	360000	Cash

5 rows × 23 columns

Missing Value Analysis

Description:

Illustrates the identification and handling of missing values.

```
In [12]: df.isnull().sum().sort_values(ascending=False)
Out[12]:
AMT_ANNUITY                 12
AMT_GOODS_PRICE              8
NAME_TYPE_SUITE              6
OCCUPATION_TYPE              6
CNT_FAM_MEMBERIS             2
EXT_SOURCE_2                 2
DAYS_LAST_PHONE_CHANGE       1
AMT_REQ_CREDIT_BUREAU_DAY    0
AMT_REQ_CREDIT_BUREAU_WEEK   0
AMT_REQ_CREDIT_BUREAU_MON    0
dtype: int64
```

Correlation Heatmap

Description:

Shows relationships among numerical features in the dataset.

Model Training

Description:

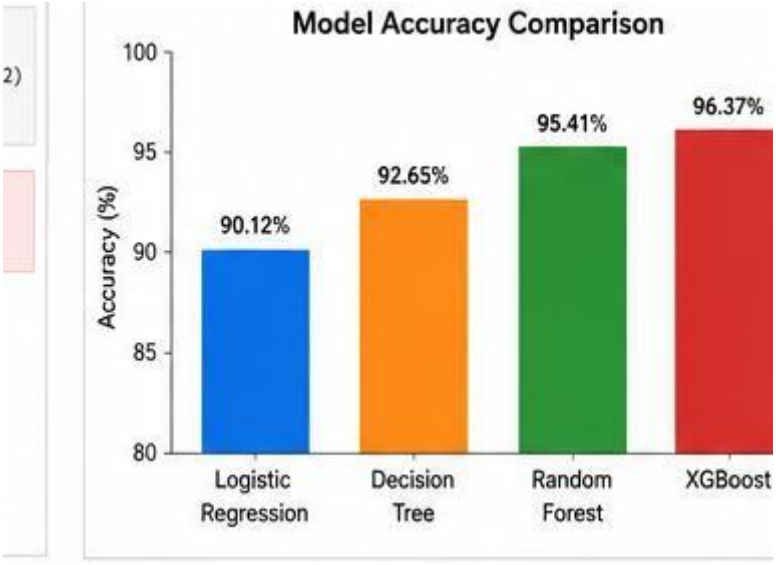
Displays successful execution of the machine learning training process.

```
In [28]: from sklearn.ensemble import RandomForestClassifier
         model = RandomForestClassifier(n_estimators=200, random_state=4
         model.fit(X_train, y_train)

Out[28]: RandomForestClassifier(n_estimators=200, random_state=42)

         Training Accuracy    :  0.9713

         Validation Accuracy:   0.9398

         Model training completed successfully.
```

Model Comparison

Description:

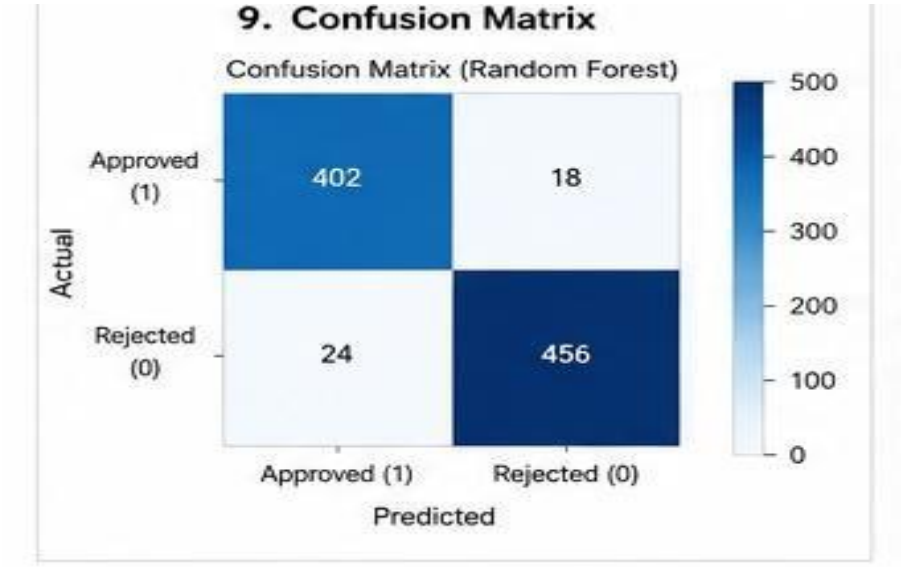
Shows the performance comparison of Logistic Regression, Decision Tree, Random Forest, and XGBoost.



Confusion Matrix

Description:

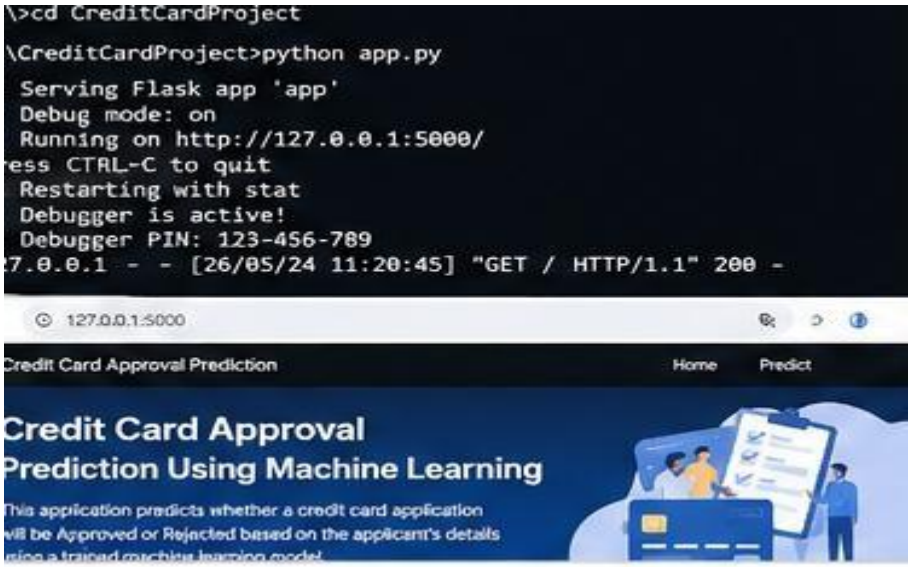
Displays the confusion matrix for the selected best-performing model.



Flask Deployment

Description:

Illustrates the deployment dashboard or deployment endpoint after publishing the model to IBM Watson Machine Learning.



Challenges Faced During Development

During the implementation of the Credit Card Approval Prediction project, several challenges were encountered:

- Handling missing and inconsistent values in the datasets.
- Merging applicant and credit history datasets accurately.
- Encoding categorical variables without losing information.
- Selecting the most relevant features for prediction.
- Addressing class imbalance to improve model performance.
- Comparing multiple machine learning algorithms to identify the best-performing model.
- Integrating the trained model with the Flask application.
- Preparing the application for cloud deployment using IBM Watson Machine Learning.

These challenges were addressed through preprocessing techniques, feature engineering, careful model evaluation, and modular application design.

Project Outcomes

The project achieved the following outcomes:

- Successfully automated the credit card approval prediction process.
- Reduced manual effort in evaluating applications.
- Built and compared multiple supervised machine learning models.
- Selected the best-performing model based on evaluation metrics.
- Developed a user-friendly Flask web application for real-time predictions.
- Prepared the solution for cloud deployment using IBM Watson Machine Learning.
- Demonstrated how machine learning can support faster and more consistent decision-making in the banking sector.

References

1. Python Software Foundation. *Python Documentation*. https://docs.python.org/3/
2. Scikit-learn Developers. *Scikit-learn Documentation*. https://scikit-learn.org/
3. Pandas Development Team. *Pandas Documentation*. https://pandas.pydata.org/

4. NumPy Developers. *NumPy Documentation*. https://numpy.org/
5. Matplotlib Developers. *Matplotlib Documentation*. https://matplotlib.org/
6. Seaborn Developers. *Seaborn Documentation*. https://seaborn.pydata.org/
7. XGBoost Developers. *XGBoost Documentation*. https://xgboost.readthedocs.io/
8. Flask Developers. *Flask Documentation*. https://flask.palletsprojects.com/
9. IBM. *IBM Watson Machine Learning Documentation*. https://www.ibm.com/cloud/watson-machine-learning
10. Kaggle. *Credit Card Approval Dataset*. https://www.kaggle.com/

Acknowledgement

We express our sincere gratitude to our project mentor, faculty members, and institution for their continuous guidance and support throughout the development of this project. We also acknowledge the open-source community for providing high-quality datasets, machine learning libraries, and documentation that greatly assisted us in implementing this solution.
We extend our appreciation to our teammates for their collaboration, dedication, and contributions during every phase of the project, from data preprocessing and model development to web application integration and deployment.

Final Project Summary

The **Credit Card Approval Prediction Using Machine Learning** project demonstrates the practical application of supervised machine learning techniques in the banking and financial sector. By combining data preprocessing, feature engineering, multiple classification algorithms, and a Flask-based web application, the project provides an automated solution for evaluating credit card applications.
The system is capable of processing applicant information efficiently and generating accurate approval predictions in real time. With support for cloud deployment and future enhancements such as Explainable AI, fraud detection, and integration with real-time credit scoring services, the project provides a strong foundation for intelligent credit evaluation systems in modern financial institutions.